The EAOS methodology defines a structured lifecycle for
Designing, deploying, and governing enterprise-grade AI systems

Project: Standard Framework Project

0.1.2 Implementation Guide

Phase: 0. SaaS Foundation

[산출물 ID: 0.1.2 Implementation Guide.docx]

① 선행 조건: ▦ Framework 문서를 먼저 읽고 설계 결정 내용을 이해한 후 이 문서를 따른다.
② 이 문서의 역할: 구현 순서, 파일 구조, 핵심 코드. 설계 이유 → Framework | 규칙 → Quick Reference

문서 번호	0.1.2 Implementation Guide.docx(How to Build)
버전	v1.0
작성일	2026
작성자	KY Song
분류	CONFIDENTIAL
적용 대상	Standard Framework 프로젝트 전체

Table of Contents

전체 구현 흐름 3
Phase 1 — 인프라 기반 4
Step 1-1. docker-compose.yml 4
Step 1-2. .env.example 4
Step 1-3. Makefile 5
Phase 2 — Backend 공통 기반 6
Step 2-1. db/base.py (반드시 가장 먼저) 6
Step 2-2. core/config.py 6
Step 2-3. core/logging.py 7
Step 2-4. db/session.py 8
Step 2-5. Alembic 초기화 (최초 1회만) 8
Step 2-6. models/base.py 8
Step 2-7. schemas/base.py 9
Step 2-8. core/exceptions.py 9
Step 2-9. core/security.py 9
Step 2-10. core/cache.py 10
Step 2-11. utils/datetime.py · utils/pagination.py 10
Step 2-12. api/deps.py 10
Step 2-13. core/audit.py · Step 2-14. main.py 11
Phase 3 — 인증·테넌트 도메인 12
Step 3-1. User 모델 핵심 컬럼 12
Step 3-2. Organization + Step 3-3. 인증 서비스 12
Phase 4 — Frontend 공통 기반 13
Step 4-1. tsconfig.json (반드시 가장 먼저) 13
Step 4-2. .eslintrc.json + .prettierrc 13
Step 4-3. next.config.ts · Step 4-4. types/ 13
Step 4-5. constants/api.ts 14
Step 4-6. lib/axios.ts — 핵심 동작 14
Step 4-7. stores/authStore.ts · Step 4-8. lib/queryClient.ts 14
Step 4-9. hooks/ · Step 4-10. 공통 컴포넌트 15
Step 4-11. app/providers.tsx · Step 4-12. tailwind.config.ts 15
Phase 5 — AI Agent 도메인 16
Step 5-1. Agent 모델 핵심 컬럼 16
Step 5-2. Agent 실행 플로우 · Step 5-3. SSE 스트리밍 16
Phase 6 — 통합 검증 및 CI/CD 17
통합 테스트 시나리오 17
전체 구현 체크리스트 18
Phase 1 — 인프라 18
Phase 2 — Backend 공통 기반 18
Phase 3 — 인증·테넌트 도메인 18
Phase 4 — Frontend 공통 기반 19
Phase 5 — AI Agent 도메인 19
Phase 6 — 통합 검증 19
주의사항 20
주요 주의사항 요약 20
승인 (Approval) 21

전체 구현 흐름

단계	항목	세부 내용
Phase 1	인프라 기반	Docker Compose · 환경 설정 · Makefile
Phase 2	Backend 공통 기반	db/base → config → logging → session → Alembic → models → schemas → exceptions → security → cache → utils → deps → main
Phase 3	인증·테넌트 도메인	users · organizations · auth (login/refresh/logout)
Phase 4	Frontend 공통 기반	tsconfig → types → constants → axios → stores → hooks → components → providers → layout → tailwind
Phase 5	AI Agent 도메인	AI Agent 도메인 Agent 모델 · 실행 서비스 · Guardrail · SSE 스트리밍
Phase 6	통합 검증·CI/CD	통합 검증·CI/CD 통합 테스트 · GitHub Actions · 헬스체크

각 Phase는 이전 Phase 완료 후 시작. Phase 내부 Step도 번호 순서를 반드시 지킨다.

Phase 1 — 인프라 기반

Step 1-1. docker-compose.yml

```yaml
services:
  db:
    image: postgres:16
    healthcheck:
      test: [´CMD-SHELL´, ´pg_isready -U ´]
      interval: 10s    retries: 5
  redis:
    image: redis:7-alpine
    healthcheck:   test: [´CMD´, ´redis-cli´, ´ping´]
  backend:
    build: ./backend
    depends_on:
      db: { condition: service_healthy }
      redis: { condition: service_healthy }
    volumes:
      - ./backend:/app    # 핫 리로드
  frontend:
    build: ./frontend
    depends_on: [backend]
volumes:
  postgres_data:
```

Step 1-2. .env.example

변수	예시 값	중요도
ENVIRONMENT	local / dev / staging / prod	필수
SECRET_KEY	(32 자 이상 랜덤)	필수 — 미설정 시 앱 기동 거부
DATABASE_URL	postgresql+asyncpg://user:pw@db:5432/db	필수
REDIS_URL	redis://redis:6379/0	필수
OPENAI_API_KEY	sk-...	AI 기능 사용 시
STRIPE_SECRET_KEY	sk_live_...	결제 기능 사용 시
NEXT_PUBLIC_API_URL	http://localhost:8000	Frontend 필수

Step 1-3. Makefile

명령어	동작
make up	docker compose up -d
make migrate	alembic upgrade head
make shell-be	Backend 컨테이너 bash
make shell-db	PostgreSQL psql 접속
make test-be	pytest tests/ -v
make lint-be	ruff check . && mypy app/

✅ Phase 1 검증: make up 실행 후 모든 컨테이너가 healthy 상태인지 확인

Phase 2 — Backend 공통 기반

이 Phase 의 파일들은 모든 도메인이 의존하는 기반이다. 반드시 Phase 3 보다 먼저 완성해야 한다.

Step 2-1. db/base.py (반드시 가장 먼저)

```
from sqlalchemy.orm import DeclarativeBase


class Base(DeclarativeBase):
    pass
```

왜 별도 파일? Alembic env.py 가 Base.metadata 를 독립적으로 import 해야 함.
모든 ORM 모델 파일에서 이 Base 를 상속한다. 순환 참조 방지의 핵심 분리점.

Step 2-2. core/config.py

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")
    environment: Literal["local","dev","staging","prod"] = "local"
    secret_key: str              # 필수, 32 자 미만 시 validator 에서 거부
    database_url: str            # 필수
    redis_url: str = "redis://localhost:6379/0"
    openai_api_key: str = ""
    agent_max_iterations: int = 10
    agent_timeout_seconds: int = 120


    @field_validator("secret_key")
    def secret_key_must_be_long(cls, v):
        if len(v) < 32: raise ValueError("SECRET_KEY must be at least 32 chars")
        return v


@lru_cache()
def get_settings() -> Settings: return Settings()
settings = get_settings()
```

Step 2-3. core/logging.py

```
import structlog


_SENSITIVE_KEYS = {"password","token","api_key","secret","card_number"}


def _mask_sensitive_fields(logger, method, event_dict):
    for key in list(event_dict.keys()):
        if key.lower() in _SENSITIVE_KEYS:
            event_dict[key] = "***MASKED***"
    return event_dict


def setup_logging(debug=False):
    structlog.configure(
        processors=[
            structlog.contextvars.merge_contextvars,
            structlog.stdlib.add_log_level,
            structlog.processors.TimeStamper(fmt="iso"),
            _mask_sensitive_fields,
            structlog.processors.JSONRenderer(),
        ],
        ...
    )


def get_logger(name: str): return structlog.get_logger(name)
```

Step 2-4. db/session.py

```
engine = create_async_engine(
    settings.database_url,
    pool_size=settings.db_pool_size,
    max_overflow=settings.db_max_overflow,
)
AsyncSessionLocal = async_sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)

# 현재 요청의 테넌트 ID — 자동 필터링에 사용
current_tenant_id: ContextVar[UUID | None] = ContextVar("current_tenant_id", default=None)

async def get_db() -> AsyncSession:
    async with AsyncSessionLocal() as session:
        yield session
```

Step 2-5. Alembic 초기화 (최초 1 회만)

```
# backend 컨테이너 안에서 실행
alembic init app/db/migrations
```

파일	수정 내용
alembic.ini	sqlalchemy.url = (빈값) — env.py에서 동적 주입
migrations/env.py	settings.database_url에서 +asyncpg 제거하여 설정
migrations/env.py	target_metadata = Base.metadata
migrations/env.py	모든 도메인 모델 import — 새 모델 추가 시마다 여기에 추가

> ⚠□ env.py에서 모델 import를 빠뜨리면 해당 테이블이 마이그레이션에서 누락된다.

Step 2-6. models/base.py

```
class BaseModel(Base):              # id(UUID) + created_at + updated_at
class SoftDeleteMixin:              # deleted_at + deleted_by + is_deleted
class TenantModel(BaseModel, SoftDeleteMixin):
    organization_id: UUID           # NOT NULL + index — 모든 테넌트 모델 필수
    created_by: UUID | None
```

Step 2-7. schemas/base.py

```
class ApiResponse(BaseModel, Generic[T]):          # 성공 단건
    success: bool = True
    data: T | None = None

class PaginatedData(BaseModel, Generic[T]):
    items: list[T]
    total: int
    page: int
    size: int
    total_pages: int

class ErrorDetail(BaseModel):                       # 에러
    code: str            # ERR_AUTH_002
    message: str
    request_id: str | None = None
```

Step 2-8. core/exceptions.py

예외 클래스	에러 코드	HTTP
UnauthorizedException	ERR_AUTH_001	401
TokenExpiredException	ERR_AUTH_002	401
ForbiddenException	ERR_AUTH_004	403
PlanLimitException	ERR_AUTH_005	403
NotFoundException	ERR_DATA_001	404
AgentGuardrailException	ERR_AGENT_003	422
AgentTimeoutException	ERR_AGENT_002	503

Step 2-9. core/security.py

```
pwd_context = CryptContext(schemes=["bcrypt"], bcrypt__rounds=12)


def hash_password(password: str) -> str
def verify_password(plain: str, hashed: str) -> bool
def create_access_token(user_id, org_id, role, plan) -> str     # 만료 15 분
def create_refresh_token(user_id) -> tuple[str, str]            # (token, jti) 30 일
def decode_token(token: str) -> dict
def hash_api_token(token: str) -> str                                # SHA-256
```

Step 2-10. core/cache.py

```
# 싱글톤 — 앱 시작 시 1회만 생성
_redis_client: redis.Redis | None = None

async def get_redis() -> redis.Redis: ...
async def cache_get(key: str) -> Any | None: ...
async def cache_set(key: str, value: Any, ttl: int = 300) -> None: ...
async def cache_delete(key: str) -> None: ...
async def close_redis() -> None: ...      # lifespan 종료 시 호출

# 캐시 키 규칙: {namespace}:{entity}:{id}
# 예: login_attempts:user@email.com
#       refresh_token:{user_id}:{jti}
```

Step 2-11. utils/datetime.py · utils/pagination.py

함수	설명
now_utc() -> datetime	현재 UTC 시각 반환 (timezone-aware)
to_kst(dt) -> datetime	UTC → KST 변환
PaginationParams(page, size)	page 최소 1, size 최대 100 자동 보정
paginate(session, query, params) -> dict	전체 count + 페이지 데이터 반환

Step 2-12. api/deps.py

```
@dataclass
class TenantContext:
    user_id: UUID
    org_id: UUID
    role: str      # owner / admin / member / viewer
    plan: str      # free / starter / pro / enterprise

async def get_tenant_context(token = Depends(oauth2_scheme)) -> TenantContext:
    payload = decode_token(token)        # 실패 시 TokenExpiredException
    current_tenant_id.set(org_id)        # 자동 테넌트 필터 주입
    return TenantContext(...)

def require_role(*roles): ...
def require_plan(*plans): ...
```

Step 2-13. core/audit.py · Step 2-14. main.py

```
# main.py 핵심 구조
@asynccontextmanager
async def lifespan(app: FastAPI):
    setup_logging(debug=settings.debug)    # 시작
    yield
    await engine.dispose()                 # 종료
    await close_redis()

app = FastAPI(lifespan=lifespan,
    docs_url="/docs" if not settings.is_production else None)

# 전역 예외 핸들러 등록
@app.exception_handler(AppException)
async def app_exception_handler(request, exc):
    return JSONResponse(status_code=exc.status_code,
        content={"success": False, "error": exc.detail})

@app.get("/health")
async def health(): return {"status": "ok"}
```

✅ Phase 2 검증: curl http://localhost:8000/health → {"status":"ok"}

Phase 3 — 인증 · 테넌트 도메인

도메인 구조 (모든 도메인에 동일 적용)

```
domains/{domain}/
├── models.py       DB 모델 (TenantModel 상속)
├── schemas.py      Pydantic 입출력 스키마
├── repository.py   DB CRUD — 비즈니스 로직 없음
├── service.py      비즈니스 로직 — DB 직접 접근 없음
└── router.py       HTTP 엔드포인트 — 서비스 호출만
```

Step 3-1. User 모델 핵심 컬럼

컬럼	타입	설명
email	VARCHAR(255) UNIQUE NOT NULL	소셜 로그인 시에도 수집
password_hash	VARCHAR NULLABLE	소셜 로그인 시 NULL
status	ENUM(active/inactive/suspended/deleted)	Soft Delete 와 별개
locale	VARCHAR(10) DEFAULT "ko"	다국어 표시 기준
timezone	VARCHAR(50) DEFAULT "Asia/Seoul"	시간대 표시 기준

Step 3-2. Organization + Step 3-3. 인증 서비스

인증 서비스 로그인 플로우	순서
Redis 에서 로그인 실패 횟수 확인 (5 회 초과 시 거부)	1
이메일로 사용자 조회 + bcrypt 비밀번호 검증	2
조직 멤버 정보로 role, plan 조회	3
Access Token (15 분) + Refresh Token (30 일) 발급	4
Refresh Token jti 를 Redis 에 저장 (TTL 30 일)	5
로그인 성공 시 Redis 실패 횟수 초기화	6

마이그레이션: alembic revision --autogenerate -m "add_users_organizations" && alembic upgrade head

Phase 4 — Frontend 공통 기반

Step 4-1. tsconfig.json (반드시 가장 먼저)

```
{
  "compilerOptions": {
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "paths": {
      "@/*":                  ["./src/*"],
      "@/components/*": ["./src/components/*"],
      "@/lib/*":          ["./src/lib/*"],
      "@/types/*":        ["./src/types/*"],
      "@/hooks/*":        ["./src/hooks/*"],
      "@/stores/*":       ["./src/stores/*"],
      "@/constants/*":  ["./src/constants/*"]
    }
  }
}
```

> path alias(@/)를 설정하지 않으면 이후 모든 import 가 상대경로가 되어 코드가 지저분해진다.

Step 4-2. .eslintrc.json + .prettierrc

항목	설정값
ESLint extends	next/core-web-vitals + next/typescript
@typescript-eslint/no-explicit-any	error — any 사용 시 CI 실패
no-console	warn
Prettier printWidth	100 자
Prettier singleQuote	true

Step 4-3. next.config.ts · Step 4-4. types/

타입	위치	대응하는 Backend
ApiResponse<T>	types/api.ts	schemas/base.py ApiResponse
PaginatedData<T>	types/api.ts	schemas/base.py PaginatedData
ApiError	types/api.ts	exceptions.py ErrorDetail
User, Organization	types/auth.ts	domains/users/models.py
MemberContext	types/auth.ts	api/deps.py TenantContext

Step 4-5. constants/api.ts

```
export const ENDPOINTS = {
  auth:    { login, refresh, logout, register },
  users:   { me, update },
  organizations: { list, detail: (id) => ..., members: (id) => ... },
  agents: { list, detail, run, stream },
} as const;
```

Step 4-6. lib/axios.ts — 핵심 동작

인터셉터	동작
요청 인터셉터	Zustand 에서 Access Token 읽어 Authorization 헤더 자동 첨부
응답 인터셉터 (401)	Access Token 만료 시 /auth/refresh 자동 호출 후 원래 요청 재시도
응답 인터셉터 (Refresh 실패)	Zustand 초기화 + /login 리다이렉트
withCredentials: true	HttpOnly Cookie (Refresh Token) 자동 포함

> ⚠□ Access Token 은 절대 localStorage 에 저장하지 않는다. XSS 취약점으로 탈취 가능.

Step 4-7. stores/authStore.ts · Step 4-8. lib/queryClient.ts

```
// authStore — Zustand 메모리 저장 (localStorage 금지)
const useAuthStore = create<AuthState>((set) => ({
    accessToken: null,
    user: null,
    setAccessToken: (token) => set({ accessToken: token }),
    logout: () => set({ accessToken: null, user: null }),
}))

// queryClient
export const queryClient = new QueryClient({
    defaultOptions: {
        queries: { staleTime: 5분, retry: 1, refetchOnWindowFocus: false },
        mutations: { retry: 0 },
    },
})
```

Step 4-9. hooks/ · Step 4-10. 공통 컴포넌트

항목	내용
useAuth.ts	useMe() — GET /users/me, useLogout() — POST /auth/logout
usePagination.ts	useState 기반 page/size 관리, goToPage(), reset()
LoadingSpinner	size(sm/md/lg) Props, Tailwind animate-spin
ErrorBoundary	Class Component, fallback props 지원

Step 4-11. app/providers.tsx · Step 4-12. tailwind.config.ts

```
// providers.tsx
<ErrorBoundary>
  <QueryClientProvider client={queryClient}>
    {children}
    {dev && <ReactQueryDevtools />}
  </QueryClientProvider>
</ErrorBoundary>

// tailwind.config.ts 디자인 토큰
colors: {
  primary: { 600: "#2563EB", 700: "#1D4ED8" },
  agent:   { bg: "#F0F9FF", border: "#BAE6FD" },
}
fontFamily: { sans: ["Pretendard","Inter"], mono: ["JetBrains Mono"] }
```

Phase 5 — AI Agent 도메인

Step 5-1. Agent 모델 핵심 컬럼

컬럼	타입	설명
agent_type	ENUM(general/analyst/researcher/coder/workflow/custom)	유형별 실행 전략
tools	JSON DEFAULT []	허용된 Tool 목록
max_iterations	INTEGER DEFAULT 10	무한 루프 방지
system_prompt	TEXT NULLABLE	Agent 역할 지시

Step 5-2. Agent 실행 플로우 · Step 5-3. SSE 스트리밍

> 실행 서비스 순서:
> 1. Feature Flag (ai_agent_enabled + plan)
> 2. Input Guardrail (길이 제한 · Moderation · 프롬프트 인젝션)
> 3. AgentRun 레코드 생성 (status=running)
> 4. asyncio.wait_for(timeout=120 초)
> 5. Output Guardrail
> 6. AgentRun 업데이트 (status=completed)
> 7. 감사 로그
>
> SSE 이벤트: token | tool_start | tool_end | done | error
> nginx 버퍼링 비활성화: X-Accel-Buffering: no 헤더 필수

Phase 6 — 통합 검증 및 CI/CD

통합 테스트 시나리오

테스트 항목	기대 결과
POST /api/v1/auth/register	201 Created
POST /api/v1/auth/login (올바른 자격증명)	200 + access_token
GET /api/v1/users/me (Bearer 포함)	200 + 사용자 정보
GET /api/v1/users/me (token 없음)	401 ERR_AUTH_001
POST /api/v1/auth/refresh	200 + 새 access_token
POST /api/v1/auth/logout 후 재사용	401
POST /api/v1/agents (pro 플랜)	201 Created
POST /api/v1/agents (free 플랜)	403 ERR_AUTH_005
GET /health	200 {"status":"ok"}

전체 구현 체크리스트

Phase 1 — 인프라

- ☐ docker-compose.yml (DB + Redis + Backend + Frontend + healthcheck)
- ☐ docker-compose.override.yml (로컬 개발 전용 오버라이드)
- ☐ .env.example 작성 + .gitignore 확인 (.env 커밋 금지)
- ☐ .env 복사 후 실제 값 입력
- ☐ Makefile 작성 (up / down / migrate / shell-be / test / lint)
- ☐ make up → 모든 컨테이너 healthy 확인

Phase 2 — Backend 공통 기반

- ☐ db/base.py (DeclarativeBase — 반드시 가장 먼저)
- ☐ core/config.py (Settings, field_validator)
- ☐ core/logging.py (structlog JSON, 민감 정보 마스킹)
- ☐ db/session.py (async engine, current_tenant_id ContextVar)
- ☐ Alembic 초기화 + alembic.ini + env.py (Base.metadata 설정)
- ☐ models/base.py (BaseModel · SoftDeleteMixin · TenantModel)
- ☐ schemas/base.py (ApiResponse · PaginatedResponse · ErrorResponse)
- ☐ core/exceptions.py (AppException + 에러 코드 전체)
- ☐ core/security.py (JWT · bcrypt cost=12)
- ☐ core/cache.py (Redis 싱글톤 · cache_get/set/delete · close_redis)
- ☐ utils/datetime.py (now_utc · to_kst)
- ☐ utils/pagination.py (PaginationParams · paginate())
- ☐ api/deps.py (TenantContext · require_role · require_plan)
- ☐ core/audit.py (create_audit_log)
- ☐ main.py (lifespan · CORS · 전역 예외 핸들러 · /health)

Phase 3 — 인증 · 테넌트 도메인

- ☐ domains/users/models.py
- ☐ domains/organizations/models.py
- ☐ env.py에 모델 import 추가 → 마이그레이션 생성 + 적용
- ☐ domains/auth/service.py (로그인 · Refresh Rotate · 로그아웃)
- ☐ domains/auth/router.py (/login · /refresh · /logout)
- ☐ domains/users/router.py (GET/PATCH /users/me)
- ☐ domains/organizations/router.py (CRUD · 멤버 초대/관리)

Phase 4 — Frontend 공통 기반

- ☐ tsconfig.json (path alias @/ — 반드시 가장 먼저)
- ☐ .eslintrc.json + .prettierrc
- ☐ next.config.ts (rewrites 프록시)
- ☐ types/api.ts (ApiResponse · PaginatedResponse · ApiError)
- ☐ types/auth.ts (User · Organization · MemberContext)
- ☐ constants/api.ts (ENDPOINTS)
- ☐ lib/axios.ts (인터셉터 · Refresh 자동 재발급)
- ☐ stores/authStore.ts (Zustand · 메모리 저장)
- ☐ lib/queryClient.ts (React Query)
- ☐ hooks/useAuth.ts · hooks/usePagination.ts
- ☐ components/common/LoadingSpinner.tsx
- ☐ components/common/ErrorBoundary.tsx
- ☐ components/ui/ (Button · Input · Modal · Toast · Table · Skeleton)
- ☐ app/providers.tsx (QueryClientProvider + ErrorBoundary)
- ☐ app/layout.tsx (Providers 감싸기)
- ☐ tailwind.config.ts (디자인 토큰)
- ☐ app/middleware.ts (인증 필요 라우트 보호)

Phase 5 — AI Agent 도메인

- ☐ domains/agents/models.py (Agent · AgentRun · AgentMemory)
- ☐ env.py 에 Agent 모델 import 추가 + 마이그레이션
- ☐ core/guardrails.py (Input · Execution · Output)
- ☐ domains/agents/service.py (실행 플로우 · Guardrail 통합)
- ☐ domains/agents/router.py (CRUD · /run · /run/stream)
- ☐ components/agents/AgentChat.tsx (SSE 수신 · 스트리밍)
- ☐ components/agents/AgentStatus.tsx (실행 상태 표시)

Phase 6 — 통합 검증

- ☐ make up → 전체 컨테이너 healthy
- ☐ curl :8000/docs → Swagger UI 접속 확인
- ☐ curl :3000 → Frontend 접속 확인
- ☐ tsc --noEmit → TypeScript 오류 0 개
- ☐ npm run lint → ESLint 오류 0 개
- ☐ mypy app/ → 타입 오류 0 개
- ☐ 통합 테스트 시나리오 9 개 전체 통과
- ☐ 브라우저 Network → API CORS 연결 확인
- ☐ .github/workflows/ci.yml 구성
- ☐ README.md 작성 (로컬 실행 방법)

주의사항

의존성 순서를 반드시 지켜야 하는 이유

Python 모듈은 import 할 때 의존하는 모듈이 먼저 존재해야 한다. 순서를 무시하면 ImportError 또는 순환 참조 오류가 발생한다.

```
db/base.py → core/config.py → core/logging.py → db/session.py
  → models/base.py → schemas/base.py → core/exceptions.py
  → core/security.py → core/cache.py → utils/* → api/deps.py → main.py
```

주요 주의사항 요약

항목	내용
.env 파일 관리	.env 는 Git 커밋 절대 금지. .env.example 은 키만 포함하여 커밋
Alembic env.py	새 모델 추가 시마다 env.py 에 import 를 추가해야 마이그레이션 감지
Alembic URL	asyncpg 드라이버는 Alembic 과 호환 안 됨. env.py 에서 +asyncpg 제거
Redis 클라이언트	redis.asyncio 사용. 동기 클라이언트와 혼용 금지
캐시 키 네이밍	{namespace}:{entity}:{id} 형태로 통일
Access Token 저장	localStorage 절대 금지. Zustand 메모리에만 저장
React 상태 관리	서버 데이터 → React Query. UI 상태(모달 등) → useState
Git 커밋 단위	각 Phase 완료 시마다 커밋으로 변경 이력 관리

승인 (Approval)

구분	내용	서명	날짜
작성자			
검토자 (AI Architect)			
승인자 (Business Owner)			
버전	v1.0		